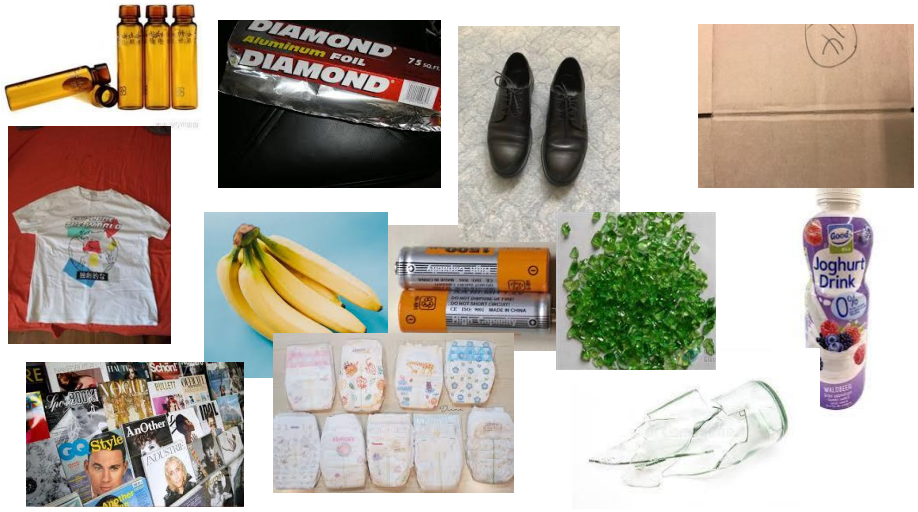
Machine Learning Project work: “Waste type identification”

Mario Vento and Diego Gragnaniello

[Link to the training dataset]



The task to address

- The goal is to recognize a broad class of images of waste objects in diverse conditions

The task to address

- The goal is to recognize a broad class of images of waste objects in diverse conditions:
  - Natural photos of different resolution
  - Preprocessed photos (e.g., background removed)
  - Multiple objects (of the same class) in the photo
  - Some class is not fully represented by the objects in the training set (i.e., some waste objects may not be in the training set, but this is normal)

The task to address

- The goal is to recognize a broad class of images of waste objects in diverse conditions:
  - 4 classes

Organic

Plastic

Battery

Metal

The task to address

- The goal is to recognize a broad class of images of waste objects in diverse conditions:
  - 4 classes
  - 3 multi-classes

Clothing
- Clothes
- Shoes

Papery
- Paper
- Cardboard

Glass
- Brown
- Green
- Transparent

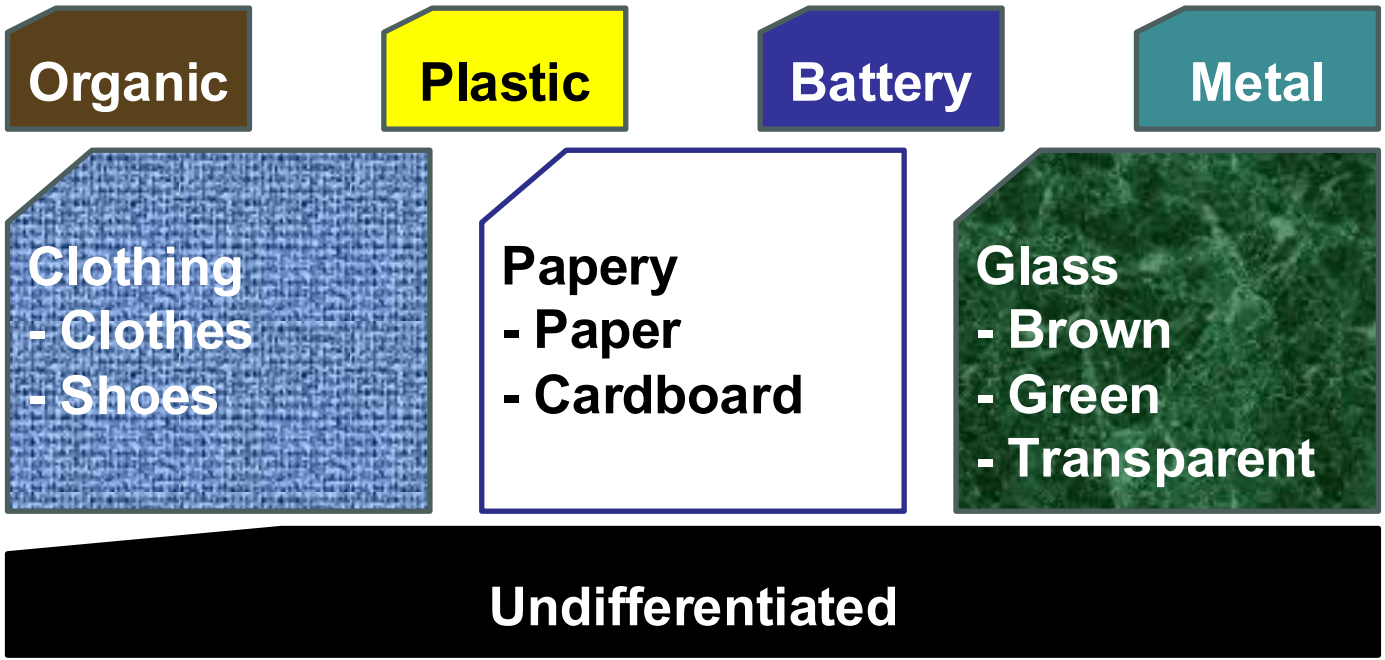
The task to address

- The goal is to recognize a broad class of images of waste objects in diverse conditions:
  - 4 classes
  - 3 multi-classes
  - 1 «other» class

Undifferentiated

The task to address

- The goal is to recognize a broad class of images of waste objects in diverse conditions:
  - 4 classes
  - 3 multi-classes
  - 1 «other» class



The task to address

- The goal is to recognize a broad class of images of waste objects in diverse conditions:
  - 4 classes
  - 3 multi-classes
  - 1 «other» class

Label 0: Battery
Label 1: Clothing
Label 2: Glass
Label 3: Metal
Label 4: Organic
Label 5: Papery
Label 6: Plastic
Label 7: Undifferentiated

Competition and rules

- Images are acquired by different cameras and in very different conditions and can be post-processed in serveral ways
- Images may not represent any kind of object for each cathegory
- You **cannot** extend the training set
- You ***can*** decide the split

IMPORTANT: Plan you experiments

- Do hypothesis on why your model is not working
  - Design an experiment to verify that specific hypothesis
  - Analyze results and comment them, thus select how to proceed
- Do a general plan of the experiments
  - Which hyperparameters you want to explore?
  - Which models/training paradigms to compare?
  - How much time does it takes?

IMPORTANT: Model training and validation is not a feed-forward process

- Alternate trainings and validations:
  - After each validation, try to understand which samples are misclassified and why
  - Change your model and/or training set to improve the performance
  - If the performance seems very good, be sure that the validation set is challenging enough
- Keep track of your countermeasures (even if not successful) for the final project presentation

What you can use

- You **cannot** extend the provided training set

- You ***can*** decide the train/val/cross-val split
- You ***can*** use data augmentation techniques

- Any PyTorch algorithm (whatever kind of classifier, including non-neural ones, preprocessing, training strategy, validation), **BUT**:
  - You must be able to explain what you have used

  - It must be **runnable with PyTorch**
    - inside **Google Colab;**
    - in **test** using **less than 4 GB GPU RAM;**
    - in **training** using **less than 5 GB GPU RAM;**

Model evaluation (1 of 2)

Performance will be evaluated on a **private test set** measuring the overall:

- Balanced accuracy, i.e., the aritmetic mean among True Positive Rates (TPR) of the 8 classes:

Bal.Acc = (TPR_battery+TPR_clothing +TPR_ glass +
TPR_ metal +TPR_ organic +TPR_ papery+
TPR_ plastic+TPR_ undifferentiated) / 8

Model evaluation (2 of 2)

In the evaluation we will consider the resources needed by the model and its computational complexity

You have to select the best trade-off among:

- Bal.Acc performance, larger is better
- GPU memory required (during test), less is better
- Processing frame rate (during test), i.e., how fast the method can process the input sample using a certain GPU, larger is better

Share the load

- Each member of the team will be requested to submit an estimate of the individual effort contributed by all members
  - To prevent "free riders"
  - Submissions will be "blind" (each member will not see the submissions of other members)

What you must submit

1. A link via mail or Moodle to:
    1. a Google Drive folder
       (**do not create a Shared Drive**)
    2. The folder must be renamed with your team name
       (see column B of the team composition sheet)
    3. Make the link **accessible to all**
    4. **Do not add Professors** to directory members

What you must submit

2. The directory must contain:

   a) The Python Notebook to train the solution, eventually including the code to generate data using augmentation

   b) The train/validation split protocol
   (**NOT THE DATA**, but the sample list as CSV file)

   c) The files of the saved models/weights after the training

   d) The code needed to test your system
   (test script; see next slide)

   e) The full report (10 pages) and slides (5 minutes talk)

What you must submit

The test script must be a Google Colab Notebook containing:

- The code to load your trained model
- A "**predict**" function with the following specification:
  - Prototype: *predict(X)*
  - where: *X* is the tensor containing a batch of test samples; the shape of *X* is*:*
    (batch_size, rows, cols, 3); the type is uint8
  - return value: the tensor with the prediction on the batch; the shape of the return value is:
    (batch_size, 1); the type is uint8 and contains the labels

  Note1: do not change the correspondance between labels and class]
  Note 2: *batch_size* is the number of samples in the batch, and is not a fixed value; the function must work independently of this value

- The *predict* function will not see the whole test set at once
- The *predict* function must perform all the required preprocessing on the batch of test data, and the postprocessing on the results before returning them

What you must submit: the presentation

- 5-minutes presentation (slide in English):
  1. Do not restate the problem/describe the dataset
  2. Go straight to your solution
     1. The rationale to collect/organize the dataset, including the dataset split into training and validation sets.
     2. The pre-processing pipelines adopted.
     3. The network architectures compared.
     4. The training hyperparameter setting, including the loss function.
  3. Let us know what challenges you encontered, how you tried to address them, what did not work and why, what contermeasure you adopted, what takehome messages you get from the results…

What you must submit: the report

- 10-pages font 12 report (in English):

1. Do not restate the problem/describe the dataset
2. Do not chronologically present your experiments without explaining how you plan them
3. Let us understand what hypothesis you did about the problem, how you managed to solve it and the results

If you want, you can present in Italian.

IMPORTANT: Don’t forget to

- Write the **names of all the team members** in the Google Drive folder (in a text file)
- Ensure that the **link** you submit is **readable to anyone** (no authorization must be requested)
- Make sure that the **test script** is **compliant with the specification** (if you have doubts about the specification, **ask**)

DEADLINE: beginning of July (TBD)

- **Tentative** project work discussion: July, 10th (time and room to be defined)
- All team members must discuss the project (including those that book for their individual oral exam in September or later)